DJANGO

1. **For making requirement.txt**
   Pip freeze > requirements.txt

2. Create Virtual Environment:

a. python -m venv env
b. .\env\Scripts\activate

Django Installation

3. python manage.py startapp myapp
   - Cd ..
   - Pip install Django
   - Django-admin startproject projectname
   - Cd projectname
   - python manage.py runserver
4. Create App
   - python manage.py startapp app
   - in settings.py, mention “app”,
5. Templates
   - Inside app create templates folder and inside it create your html files.
   - python manage.py migrate
     - inside urls.py of app:
       - from django.urls import path
       - from . import views
   - `{% include 'footer.html' %}` ->>> used to include header or footer file in required page

6. Authentication complete Procedure:

a. In main project, urls.py:
   - path(' ', include('app.urls')),

b. in settings.py
   - STATIC_URL = "static/"
   - LOGIN_REDIRECT_URL = '/home/'
   - LOGOUT_REDIRECT_URL = '/login/'
   - Include app in setting also
   - 'django.contrib.messages.middleware.MessageMiddleware',

c. In app/urls.py
   - path("home/", views.HomeView, name="home"), # Home view
   - path("signup/", views.signup_view, name="signup"), # Signup view
   - path("login/", views.login_view, name="login"), # Login view (no leading slash)
   - path("logout/", views.logout_view, name="logout"),

d. In app/views.py

```
from django.shortcuts import render,redirect
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth import authenticate,login,logout
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from .form import StudentForm
# Create your views here.
@login_required
def HomeView(request):
    return render(request,"home.html")

def signup_view(request):
    if request.method=='POST':
        form=UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login')
        else:
            form=UserCreationForm()
    return render(request,'signup.html',locals())

def login_view(request):
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)  # Log the user in
            next_url = request.GET.get('next')  # Get the next URL from the query
parameters
            if next_url:  # If there is a next URL, redirect there
                return redirect(next_url)
            return redirect('home')  # Default redirect to home if no next URL
        else:
            messages.error(request, "Invalid username or password")
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

def logout_view(request):
    logout(request)
    messages.success(request,"Logout Successfully")
    return redirect('login')
```

e. in templates:

#signup:

```
<form method="POST">
        {% csrf_token %}
```

```
            {% for field in form %}
            <div class="form-group">
              <ul class="bullets" style="list-style-type: none; padding-left: 0; margin: 0;">
              <li> {{ field.label_tag }} </li>
              <li> {{ field }}</li>
             </ul>
            </div>
          {% endfor %}
          <br>
            <button type="submit" class="btn btn-primary">Register</button>
          </form>

          {% if messages %}
            <div class="message-container">
              {% for message in messages %}
                <div class="alert {{ message.tags }}">
                  {{ message }}
                </div>
              {% endfor %}
            </div>
          {% endif %}
<a href="{% url 'login' %}">Already have an account? Login</a>
```

 # login

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Login</button>
</form>

{% if messages %}
  <ul>
    {% for message in messages %}
      <li>{{ message }}</li>
    {% endfor %}
  </ul>
{% endif %}
<a href="{% url 'signup' %}">Don't have an account? Sign up</a>
```

#home

```
<a href="{% url 'logout' %}">Logout</a>
```

7. User Created Form

- **Pip install pillow –** for images
- **In settings.py**

- MEDIA_ROOT = BASE_DIR / 'media'
- MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

- In app/urls.py
  - from django.conf import settings
  - from django.conf.urls.static import static
  - if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
  - path('registration/',views.Studentview,name='registration')
- models.py

```
from django.db import models

# Choices for the educational programs
choices_program = [
    ('cse', "Computer Science and Engineering"),
    ('bba', "Bachelor in Business Administration")
]

class Student(models.Model):
    first_name = models.CharField(max_length=255)
    last_name = models.CharField(max_length=255)
    id = models.CharField(primary_key=True,max_length=10)  # Use AutoField
for an auto-incrementing primary key
    photo = models.ImageField(upload_to='images/')
    phone = models.CharField(max_length=15)  # Change to CharField to allow
phone number formatting
    branch = models.CharField(choices=choices_program, default='cse',
max_length=3)

    def __str__(self):
        return f"{self.first_name} {self.last_name} - {self.branch}"
```

- Forms.py

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = ['first_name', 'last_name', 'id', 'photo', 'phone',
'branch']
```

- Admin.py

```
from django.contrib import admin
from .models import Student
class StudentAdmin(admin.ModelAdmin):
  list_display = ("first_name", "last_name", "id",)
    admin.site.register(Student, StudentAdmin)
```

- Views.py

```
def Studentview(request):
    if request.method=='POST':
        form=StudentForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            messages.success(request,"saved successfully")
    else:
        form=StudentForm()
    return render(request,'registration.html',locals())
```

- Registration.html

```
<form method="POST" enctype="multipart/form-data">

        {% csrf_token %}
        {% for field in form %}
        <div class="form-group">
          <ul class="bullets" style="list-style-type: none; padding-left: 0; margin: 0;">
          <li> {{ field.label_tag }} </li>
          <li> {{ field }}</li>
         </ul>
        </div>
      {% endfor %}
      <br>
        <button type="submit" class="btn btn-primary">Register</button>
      </form>

      {% if messages %}
        <div class="message-container">
          {% for message in messages %}
            <div class="alert {{ message.tags }}">
              {{ message }}
            </div>
```

{% endfor %}

{% endif %}

User made admin

```
def Adminview(request):
    students = Student.objects.all()
    return render(request, 'admin.html', {'students': students})
```

{% for student in students %}

{{ student.id }} {{ student.first_name }} update

{% endfor %}

Update Member:

1. views.py

```
@login_required
def update_vendor(request, vendor_id):
    vendor = get_object_or_404(Vendor_Registration, id=vendor_id)

    if request.method == 'POST':
        form = VendorUpdateForm(request.POST, instance=vendor)
        if form.is_valid():
            form.save()
            messages.success(request, 'Vendor details updated successfully.')
            return redirect('vendor_list')
    else:
        form = VendorUpdateForm(instance=vendor)

    return render(request, 'update_vendor.html', {'form': form, 'vendor': vendor})
```

2. urls.py:

```
path('vendor/update/<int:vendor_id>/', views.update_vendor, name='update_vendor'),
```

3. templates/html(same as above registration form)

4. to make button for update:

```
<a href="{% url 'update_vendor' vendor.id %}" class="btn btn-danger">Update</a>
```

Delete Member or Student

```
def delete_student(request,id):
    student=get_object_or_404(Student,id=id)
    student.delete()
    return redirect('adminview')
```

Note: everything is same as above update.

To Access the person who logged in

```
@login_required
def profile(request):
    user = request.user  # Retrieve the currently logged-in user
    context = {'user': user}  # Pass user to context explicitly
    return render(request, 'profile.html', context)
```

in html page, {{user.firstname}}

Admin Panel

- py manage.py createsuperuser
- To include the Member model in the admin interface, we have to tell Django that this model should be visible in the admin interface.
- In app/admin.py

```
from .models import Member

# Register your models here.
admin.site.register(Member)
```

- set list display:
  - We can control the fields to display by specifying them in in a list_display property in the admin.py file.
  - In admin.py of app

```
from django.contrib import admin

from .models import Member

class MemberAdmin(admin.ModelAdmin):
  list_display = ("firstname", "lastname", "joined_date",)


admin.site.register(Member, MemberAdmin)
```

QuerySelect

- A QuerySet is built up as a list of objects.

- QuerySets makes it easier to get the data you actually need, by allowing you to filter and order the data at an early stage.

```
from django.shortcuts import render
from .models import Member

def testing(request):
  mydata = Member.objects.all()
  context = {'mymembers': mydata}
  return render(request, 'template.html', context)
```

//this will give all data from member table.

//template should be:

```
{% for x in mymembers %}
  <tr>
   <td>{{ x.id }}</td>
    <td>{{ x.firstname }}</td>
   <td>{{ x.lastname }}</td>
  </tr>
{% endfor %}
```

Note:

- mydata = Member.objects.all().values() -→this will return data in dictionary format
- `mydata = Member.objects.values_list('firstname')-→ to make a list`
- `mydata = Member.objects.filter(firstname='Emil').values() → to return particular row`
- `mydata = Member.objects.filter(lastname='Refsnes', id=2).values()  →AND query`
- `mydata = Member.objects.filter(firstname='Emil').values() | Member.objects.filter(firstname='Tobias').values()  →OR query`
- `mydata = Member.objects.all().order_by('firstname').values()  →ascending order`
- `mydata = Member.objects.all().order_by('-firstname').values()  →descending order`

1. Add Static File

- **Create static folder in app**
- **Create css file in static folder**
- `{% load static %}  →write this before including css file or images`
- `<link rel="stylesheet" href="{% static 'myfirst.css' %}">`
- **pip install whitenoise (to access static files)**
- `'whitenoise.middleware.WhiteNoiseMiddleware',  →include in middleware`
- `In setting.py`

```
STATIC_ROOT = BASE_DIR / 'productionfiles'

STATIC_URL = 'static/'
```

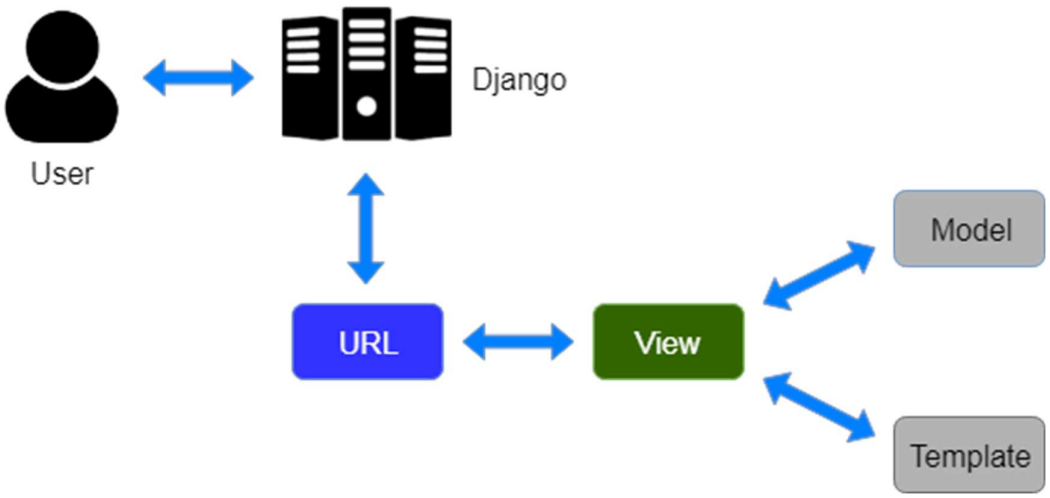
- py manage.py collectstatic

2. for images upload

- **pip install Pillow**
- **in models.py**

MVT

The MVT (Model View Template) is a software design pattern. It is a collection of three important components Model View and Template. The Model helps to handle database. It is a data access layer which handles the data.



USER MADE AUTHENTICATION

1. model.py

```
class Vendor_Registration(models.Model):
    username = models.CharField(max_length=150, unique=True)
    email = models.EmailField()
    password = models.CharField(max_length=255)
    confirm_password = models.CharField(max_length=255)

    def __str__(self):
        return self.username
```

2. forms.py

```
from django import forms
from .models import Vendor_Registration,Product,User_Registration
```

```python
class VendorForm(forms.ModelForm):
    class Meta:
        model = Vendor_Registration
        fields = ['username', 'email', 'password', 'confirm_password']
        widgets = {
            'password': forms.PasswordInput(),
            'confirm_password': forms.PasswordInput(),
        }
    def clean(self):
        cleaned_data = super().clean()
        password = cleaned_data.get("password")  # Fixed typo here
        confirm_password = cleaned_data.get("confirm_password")

        if password and confirm_password and password != confirm_password:
            raise forms.ValidationError("Passwords do not match.")

        return cleaned_data
class LoginForm(forms.Form):
    username = forms.CharField(max_length=150)
    password = forms.CharField(widget=forms.PasswordInput)
```

3. views.py

```python
def HomeView(request):
    return render(request,"home.html")


def Vendor_Registration_View(request):
    if request.method=="POST":
        form=VendorForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request,"registration successfull")
            return redirect("vendor_login")
        else:
            messages.error(request,"please correct the error below")
    else:
        form=VendorForm()
    return render(request,"vendor_registration.html",{'form':form})


def Login_View(request):
    form = LoginForm(request.POST or None)

    if request.method == 'POST' and form.is_valid():
        username = form.cleaned_data['username']
        password = form.cleaned_data['password']
        try:
            user = Vendor_Registration.objects.get(username=username)
```

```
            if user.password == password:
                request.session['vendor_id'] = user.id
                return redirect('vendor_home')
            else:
                messages.error(request, "Invalid credentials.")
        except Vendor_Registration.DoesNotExist:
            messages.error(request, "Vendor does not exist. Please register.")
    return render(request, 'vendor_login.html', {'form': form})


def Logout_View(request):
    if 'vendor_id' in request.session:
        del request.session['vendor_id']
        messages.success(request, "You have successfully logged out.")
    else:
        messages.info(request, "You are not logged in.")

    return redirect('home')
```

4. urls.py

```
path("", views.HomeView, name="home"),
    path("vendor_registration/",views.Vendor_Registration_View,name="vendor_registra
tion"),
    path("vendor_login/",views.Login_View,name="vendor_login"),
    path("logout_vendor/",views.Logout_View,name="logout_vendor"),
```

5. templates/html files are same as above. (most important part is in form and login.view)

MODEL

- Each model class maps to a single table in the database.
- Django Model is a subclass of django.db.models.Model and each field of the model class represents a database field (column).
- Django provides us a database-abstraction API which allows us to create, retrieve, update and delete a record from the mapped table.
- Model is defined in Models.py file.

For Example:

```
from django.db import models

class Employee(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

Field Name	Class	Particular
AutoField	class AutoField(**options)	It An IntegerField that automati increments.

BigAutoField	class BigAutoField(**options)	It is a 64-bit integer, much like AutoField except that it is guaranteed numbers from 1 9223372036854775807.
BigIntegerField	class BigIntegerField(**options)	It is a 64-bit integer, much like IntegerField except that it is guarantee fit numbers from 9223372036854775808 9223372036854775807.
BinaryField	class BinaryField(**options)	A field to store raw binary data.
BooleanField	class BooleanField(**options)	A true/false field. The default form wi for this field is a CheckboxInput.
CharField	class DateField(auto_now=False, auto_now_add=False, **options)	It is a date, represented in Python datetime.date instance.
DateTimeField	class DateTimeField(auto_now=False, auto_now_add=False, **options)	It is a date, represented in Python datetime.date instance.
DateTimeField	class DateTimeField(auto_now=False, auto_now_add=False, **options)	It is used for date and time, represente Python by a datetime.datetime instanc
DecimalField	class DecimalField(max_digits=None, decimal_places=None, **options)	It is a fixed-precision decimal num represented in Python by a Dec instance.
DurationField	class DurationField(**options)	A field for storing periods of time.
EmailField	class EmailField(max_length=254, **options)	It is a CharField that checks that the v is a valid email address.
FileField	class FileField(upload_to=None, max_length=100, **options)	It is a file-upload field.
FloatField	class FloatField(**options)	It is a floating-point number represe in Python by a float instance.
ImageField	class ImageField(upload_to=None, height_field=None, width_field=None, max_length=100, **options)	It inherits all attributes and methods f FileField, but also validates that uploaded object is a valid image.

IntegerField	class IntegerField(**options)	It is an integer field. Values fro 2147483648 to 2147483647 are safe i databases supported by Django.
NullBooleanField	class NullBooleanField(**options)	Like a BooleanField, but allows NUL one of the options.
PositiveIntegerField	class PositiveIntegerField(**options)	Like an IntegerField, but must be ei positive or zero (0). Values from 2147483647 are safe in all datab supported by Django.
SmallIntegerField	class SmallIntegerField(**options)	It is like an IntegerField, but only all values under a certain (datab dependent) point.
TextField	class TextField(**options)	A large text field. The default form wi for this field is a Textarea.
TimeField	class TimeField(auto_now=False, auto_now_add=False, **options)	A time, represented in Python b datetime.time instance.

Field Options	Particulars
Null	Django will store empty values as NULL in the database.
Blank	It is used to allowed field to be blank.
Choices	An iterable (e.g., a list or tuple) of 2-tuples to use as choice this field.
Default	The default value for the field. This can be a value or a call object.
help_text	Extra "help" text to be displayed with the form widget. It's us for documentation even if your field isn't used on a form.
primary_key	This field is the primary key for the model.
Unique	This field must be unique throughout the table.

Django Model Forms

```
from django import forms
from myapp.models import Student

class EmpForm(forms.ModelForm):
  class Meta:
    model = Student
    fields = "__all__"
```

in views.py

```
1. def index(request):
2.     stu = EmpForm()
3.     return render(request,"index.html",{'form':stu}
```

forms can be used to create html forms

```
from django import forms
class StudentForm(forms.Form):
  firstname = forms.CharField(label="Enter first name",max_length=50)
  lastname  = forms.CharField(label="Enter last name", max_length = 100)
```

- {{ form.as_table }} will render them as table cells wrapped in <tr> tags
- {{ form.as_p }} will render them wrapped in <p> tags
- {{ form.as_ul }} will render them wrapped in <li> tags

Django forms submit only if it contains CSRF tokens.

It uses uses a clean and easy approach to validate data.

The **is_valid()** method is used to perform validation for each field of the form, it is defined in Django Form class.

the **forms.FileField()** method is used to create a file input and submit the file to the server. While working with files, make sure the HTML form tag contains **enctype="multipart/form-data"** property.

```
1. <form method="POST" class="post-form" enctype="multipart/form-data">
2.     {% csrf_token %}
3.     {{ form.as_p }}
4.     <button type="submit" class="save btn btn-default">Save</button>
5. </form>
```

```
def index(request):
  if request.method == 'POST':
    student = StudentForm(request.POST, request.FILES)
```

Django Exception

Django core exceptions classes are defined in **django.core.exceptions** module. This module contains the following classes.

Django Exception Classes

Exception	Description
AppRegistryNotReady	It is raised when attempting to use models before the app loading process.
ObjectDoesNotExist	The base class for DoesNotExist exceptions.
EmptyResultSet	If a query does not return any result, this exception is raised.
FieldDoesNotExist	It raises when the requested field does not exist.
MultipleObjectsReturned	This exception is raised by a query if only one object is expected, but multiple objects are returned.
SuspiciousOperation	This exception is raised when a user has performed an operation that should be considered suspicious from a security perspective.
PermissionDenied	It is raised when a user does not have permission to perform the action requested.
ViewDoesNotExist	It is raised by django.urls when a requested view does not exist.
MiddlewareNotUsed	It is raised when a middleware is not used in the server configuration.
ImproperlyConfigured	The ImproperlyConfigured exception is raised when Django is somehow improperly configured.
FieldError	It is raised when there is a problem with a model field.

ValidationError	It is raised when data validation fails form or model field validation.

Django URL Resolver Exceptions

These exceptions are defined in **django.urls** module.

Exception	Description
Resolver404	This exception raised when the path passed to resolve() function does not map to a view.
NoReverseMatch	It is raised when a matching URL in your URLconf cannot be identified based on the parameters supplied.
Exception	Description
DatabaseError	It occurs when the database is not available.
IntegrityError	It occurs when an insertion query executes.
DataError	It raises when data related issues come into the database.

```
def getdata(request):
    try:
        data = Employee.objects.get(id=12)
    except ObjectDoesNotExist:
        return HttpResponse("Exception: Data not found")
    return HttpResponse(data);
```

Session

A session is a mechanism to store information on the server side during the interaction with the web application.

In Django, by default session stores in the database and also allows file-based and cache based sessions. It is implemented via a piece of middleware and can be enabled by using the following code.

Put django.contrib.sessions.middleware.SessionMiddleware in MIDDLEWARE and django.contrib.sessions in INSTALLED_APPS of settings.py file.

To set and get the session in views, we can use request.session and can set multiple times too.

The class backends.base.SessionBase is a base class of all session objects. It contains the following standard methods.

Method	Description
__getitem__(key)	**It is used to get session value.**
__setitem__(key, value)	**It is used to set session value.**
__delitem__(key)	**It is used to delete session object.**
__contains__(key)	**It checks whether the container contains the particular session object or not.**
get(key, default=None)	**It is used to get session value of the specified key.**

```
def setsession(request):
    request.session['sname'] = 'irfan'
    request.session['semail'] = 'irfan.sssit@gmail.com'
```

```
    return HttpResponse("session is set")
```

```
1.  path('ssession',views.setsession),
2.      path('gsession',views.getsession)
```

Cookie

The **set_cookie()** method is used to set a cookie and **get()** method is used to get the cookie.

The **request.COOKIES['key']** array can also be used to get cookie values.

```
from django.shortcuts import render
from django.http import HttpResponse

def setcookie(request):
  response = HttpResponse("Cookie Set")
  response.set_cookie('java-tutorial', 'javatpoint.com')
  return response
def getcookie(request):
  tutorial  = request.COOKIES['java-tutorial']
  return HttpResponse("java tutorials @: "+  tutorial);
```

```
1.  path('scookie',views.setcookie),
2.      path('gcookie',views.getcookie)
```

Django PDF

To generate PDF, we will use **ReportLab** Python PDF library that creates customized dynamic PDF.

```
1.  $ pip install reportlab
```

// views.py

```
1.  from reportlab.pdfgen import canvas
2.  from django.http import HttpResponse
3.
4.  def getpdf(request):
5.      response = HttpResponse(content_type='application/pdf')
6.      response['Content-Disposition'] = 'attachment; filename="file.pdf"'
7.      p = canvas.Canvas(response)
8.      p.setFont("Times-Roman", 55)
9.      p.drawString(100,700, "Hello, Javatpoint.")
10.     p.showPage()
11.     p.save()
```

12. **return** response

First, provide MIME (content) type as application/pdf, so that output generates as PDF rather than HTML,

Set Content-Disposition in which provide header as attachment and output file name.

Pass response argument to the canvas and drawstring to write the string after that apply to the save() method and return response.

// urls.py

1. path('pdf',views.getpdf)

Relationship model

1. **One-to-One Relationship**

```
from django.db import models

class User(models.Model):
    name = models.CharField(max_length=100)

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField()
```

2. 2. Many-to-One Relationship (ForeignKey)

```
class Author(models.Model):
    name = models.CharField(max_length=100)

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

3. 3. Many-to-Many Relationship

```
class Student(models.Model):
    name = models.CharField(max_length=100)

class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(Student)
```

PDF

- pip install WeasyPrint
- # pdfapp/views.py
-
- from django.shortcuts import render
- from django.http import HttpResponse
- from weasyprint import HTML
-
- def generate_pdf(request):
- # Render the HTML template
- html_string = render(request, 'pdfapp/template.html', {}).content.decode()
-
- # Create PDF from rendered HTML
- pdf = HTML(string=html_string).write_pdf()
-
- # Return PDF as response

- response = HttpResponse(pdf, content_type='application/pdf')
- response['Content-Disposition'] = 'inline; filename="output.pdf"'
- return response

```
# pdfapp/urls.py

from django.urls import path
from .views import generate_pdf

urlpatterns = [
    path('generate-pdf/', generate_pdf, name='generate_pdf'),
]

# myproject/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('pdfapp/', include('pdfapp.urls')),  # Include PDF app URLs
]



<!-- pdfapp/templates/pdfapp/template.html -->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PDF Report</title>
```

```
    <style>
        body { font-family: Arial, sans-serif; }
        h1 { color: #333; }
    </style>
</head>
<body>
    <h1>Welcome to PDF Generation in Django!</h1>
    <p>This PDF was generated using WeasyPrint.</p>
</body>
</html>
```